Inhalt

Protocol – TourPlanner 1
App Architecture 1
Use Cases 2
Library Decisions 2
Implemented Design Patterns 2
Unit Testing Decisions 3
Lessons Learned 3
Tracked Time 3
Git Link 3

Protocol – TourPlanner

App Architecture

The *TourPlanner App* uses **MVVM architecture (Model-View-ViewModel)** for separation of concerns:

1. **UI Layer (View)**
   - Contains all XAML views (e.g., MainWindow, AddTourWindow, EditTourLogWindow).
   - Displays data and forwards user actions to the ViewModels.

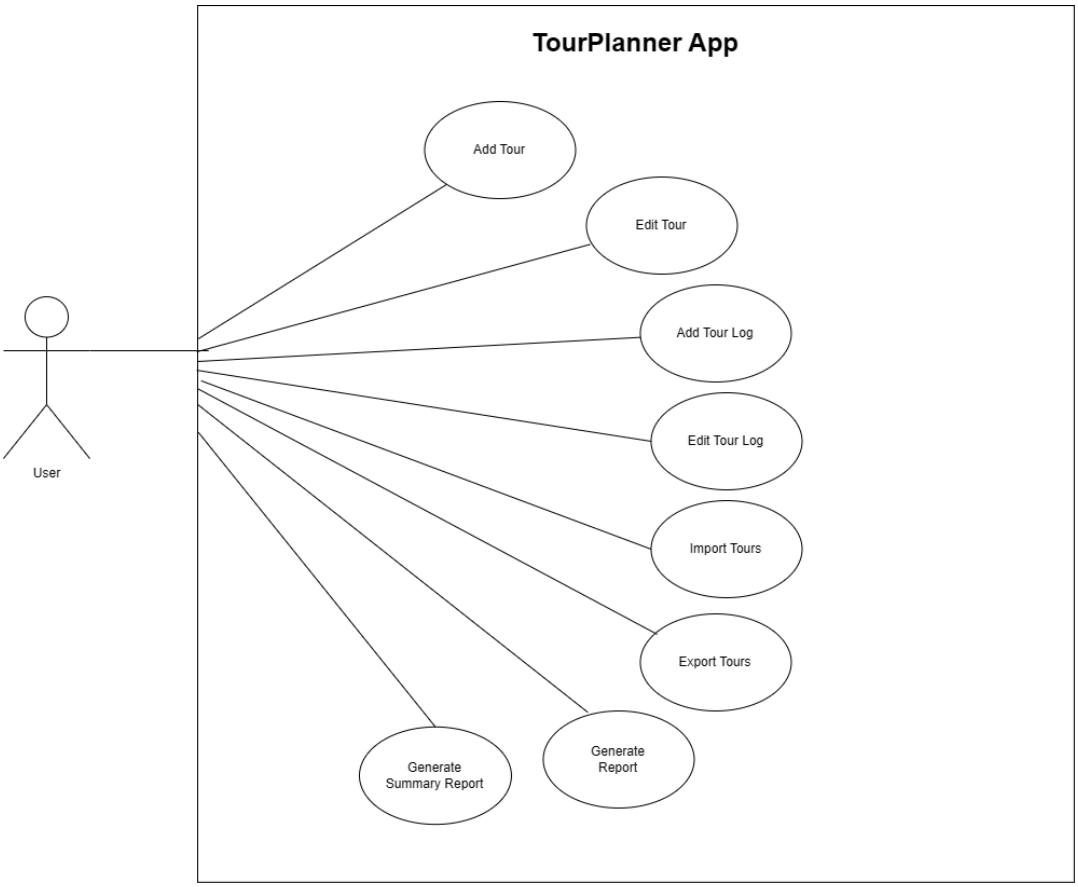
2. **ViewModel Layer**
   - Connects the UI to the business logic (e.g., MainViewModel, AddTourViewModel).
   - Implements commands like OpenAddTourWindowCommand or DeleteTourCommand.

3. **Service Layer**
   - Contains the business logic for tours and logs (e.g., TourService, TourLogService).
   - Communicates with the database using repositories.

4. **Data Access Layer (Repository)**
   - Handles all database operations using **Entity Framework Core**.
   - Saves tours and logs into a PostgreSQL database (via Npgsql.EntityFrameworkCore.PostgreSQL).

Use Cases

Use Case Diagram:



Library Decisions

- **MahApps.Metro**: for modern WPF UI styling.
- **Entity Framework Core**: as ORM for database operations.
- **log4net**: for logging and error tracking.
- **PdfSharpCore** and **SkiaSharp**: to create PDF reports and charts.
- **Microsoft.Extensions.DependencyInjection**: for dependency injection and better testability.
- **WebView2**: for integrating web-based components if needed.

Implemented Design Patterns

- **MVVM (Model-View-ViewModel):** clear separation of UI and logic.
- **Repository Pattern:** abstracts data access logic.
- **Command Pattern:** handles user actions in the UI (e.g., Add Tour, Delete Tour).
- **Dependency Injection:** enables loose coupling and easier testing.

Unit Testing Decisions

- Focus on testing:
  - **Services** (e.g., TourServiceTests, TourLogServiceTests).
  - **ViewModels** (e.g., AddTourViewModelTests).
- Test Framework: NUnit.
- Fake Repositories and Mocks are used to isolate logic from the database.
- UI is not tested; focus is on business logic and validation.

Lessons Learned

- Separating UI and logic makes the app easier to maintain and extend.
- Entity Framework Core simplifies data access but requires careful migration handling.
- MahApps.Metro helps to create a modern UI but has a learning curve.
- Unit testing for services is simple, but testing ViewModels requires careful handling of commands and bindings.

Tracked Time

We kept track of our working time by writing it down and also calculated it based on our Git repository history. The total effort is estimated to be approximately 90 hours.

Git Link

https://github.com/filiplzrvc/TourPlanner